

Лабораторная работа №4

Продвинутое использование Git

Чжан Вэньцзюнь

РУДН

2026-03-07

Цель работы

- Получить практические навыки работы с системой контроля версий Git
- Освоить модель разработки Gitflow Workflow
- Изучить принципы семантического версионирования (SemVer)
- Ознакомиться со стандартом Conventional Commits
- Настроить автоматическую генерацию CHANGELOG.md

Подготовка рабочего окружения

Для выполнения работы были установлены следующие инструменты:

- git-flow
- Node.js
- npm
- pnpm
- commitizen
- standard-changelog
- GitHub CLI

```
(base) hhl@LAPTOP-958L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ sudo apt update
[sudo] password for hhl:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Hit:5 https://cli.github.com/packages stable InRelease
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1504 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1806 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [241 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.1 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [975 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.1 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.6 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:15 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [332 kB]
```

Создание репозитория

На GitHub был создан новый репозиторий:

```
git-extended
```

Локальная инициализация:

```
git init
git remote add origin git@github.com:wenjun/git-extended

echo "# Git Extended Lab" > README.md

git add README.md
git commit -m "first commit"

git branch -M master
git push -u origin master
```

```
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ git init
Initialized empty Git repository in /home/hhl/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended
/.git/
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ git remote add
```

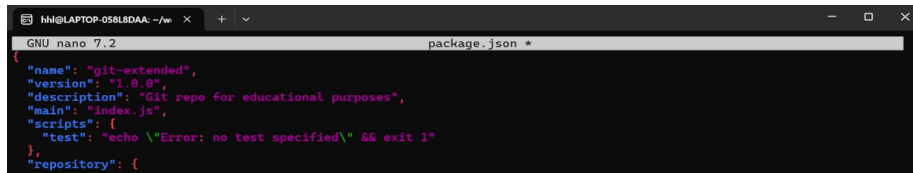
Настройка проекта Node.js

Создание файла package.json:

```
pnpm init
```

Настройка Commitizen:

```
{  
  "config": {  
    "commitizen": {  
      "path": "cz-conventional-changelog"  
    }  
  }  
}
```

A screenshot of a terminal window with a dark background. The title bar shows 'hhl@LAPTOP-0581BDAA: ~/w' and window control buttons. The terminal shows 'GNU nano 7.2' at the top left and 'package.json *' at the top right. The content of the file is a JSON object with fields for name, version, description, main, scripts, test, and repository. The 'test' field contains a command to echo an error message and exit with status 1. The 'scripts' field is an empty object. The 'repository' field is an empty object. The file is currently open in the nano text editor.

```
{  
  "name": "git-extended",  
  "version": "1.0.0",  
  "description": "Git repo for educational purposes",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "repository": {  
    "type": "git",  
    "url": "https://github.com/hhl/git-extended.git"  
  }  
}
```

Conventional Commits

Для создания стандартизированных сообщений использовалась команда:

```
git cz
```

Первый коммит был создан с типом:

```
chore
```

Преимущества использования Conventional Commits:

- единый стиль сообщений
- автоматическая генерация CHANGELOG
- удобное управление версиями

```
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0
? Select the type of change that you're committing: chore:   Other changes that don't modify src or test files
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 93 chars):
  (45) initialize package.json and commitizen config
? Provide a longer description of the change: (press enter to skip)
? Are there any breaking changes? No
```

Инициализация Gitflow

Запуск Gitflow:

```
git flow init
```

Использованы настройки:

- master — основная ветка релизов
- develop — ветка разработки
- feature/* — функциональные ветки
- release/* — ветки релизов
- hotfix/* — ветки исправлений

Отправка веток:

```
git push --all
```

```
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ git flow init
Which branch should be used for bringing forth production releases?
- master
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [hotfix/]

```

Создание релиза v1.0.0

Создание релизной ветки:

```
git flow release start 1.0.0
```

Генерация CHANGELOG:

```
standard-changelog --first-release
```

Завершение релиза:

```
git flow release finish 1.0.0
```

Публикация тегов:

```
git push --tags
```

Создание релиза на GitHub:

```
gh release create v1.0.0 -F CHANGELOG.md
```

```
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ gh release crea  
te v1.0.0 -F CHANGELOG.md
```


Работа с Feature-веткой

Создание новой функциональности:

```
git flow feature start feature_branch
```

Создание файла:

```
echo "console.log('new feature');" > new-feature.js
```

Коммит изменений:

```
git cz
```

Тип коммита:

```
feat
```

Завершение разработки:

```
git flow feature finish feature_branch
```

Создание релиза v1.2.3

Обновление версии:

```
"version": "1.2.3"
```

Обновление CHANGELOG:

```
standard-changelog
```

Завершение релиза:

```
git flow release finish 1.2.3
```

Отправка изменений:

```
git push --all  
git push --tags
```

Публикация релиза:

```
gh release create v1.2.3 -F CHANGELOG.md
```

Просмотр истории проекта

Для анализа истории использовалась команда:

```
git log --oneline --graph --all
```

Она позволяет наглядно отобразить:

- историю коммитов
- ветвление проекта
- теги релизов
- операции слияния

```
(base) hh1@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab04/git-extended$ git log --oneli
ne --graph --all
* b585064 (HEAD -> develop, origin/develop) Merge tag 'v1.2.3' into develop
|
| * 49c3cca (tag: v1.2.3, origin/master, master) Merge branch 'release/1.2.3'
|/
| * 8a968a5 chore(site): update changelog for 1.2.3
|/
|
| * 425fb2c Merge tag 'v1.0.0' into develop
| \
| * c4907da (tag: v1.0.0) Merge branch 'release/1.0.0'
|/
| * c3c5a99 chore(site): add changelog
|/
| * 5dc2f7a feat: add new feature file
|/
|
| * 7749a59 chore: initialize package.json and commitizen config
|
| * a9cbad7 first commit
```

Основные ветки:

- master
- develop

Дополнительные ветки:

- feature/*
- release/*
- hotfix/*

Преимущества Gitflow:

- разделение процессов разработки и выпуска
- удобная работа в команде
- контроль стабильности проекта
- прозрачная история изменений

Семантическое версионирование

Стандарт SemVer использует формат:

`MAJOR.MINOR.PATCH`

Где:

- MAJOR — несовместимые изменения
- MINOR — новая функциональность
- PATCH — исправление ошибок

Примеры:

`1.0.0`
`1.2.3`
`2.0.0`

Связь с Conventional Commits:

- feat → MINOR
- fix → PATCH
- BREAKING CHANGE → MAJOR

Выводы

В ходе выполнения лабораторной работы были получены практические навыки использования Git и Gitflow Workflow.

Освоены основные принципы семантического версионирования и стандарта Conventional Commits.

Настроена автоматическая генерация файла CHANGELOG.md, выполнена работа с тегами и публикацией релизов на GitHub.

Также были закреплены навыки управления ветками проекта, создания релизов и организации процесса коллективной разработки программного обеспечения.

Все поставленные задачи лабораторной работы выполнены успешно.

Спасибо за внимание!